

# SOFTWARE QUALITY ASSURANCE

HNDIT-4022

## *Academic Short Notes — Week by Week*

A structured revision guide covering Weeks 1–14: key definitions, concepts, examples and diagrams for quick learning and exam preparation.

## Contents

---

- Week 1 — Introduction to Software Testing & SQA
- Week 2 — Software Reviews
- Week 3 — Static Verification Techniques
- Week 4 — Software Testing Techniques (Black / White / Gray Box)
- Week 5 — Test Case Design & Test Cases
- Week 6 — Test Automation
- Week 7 — Performance Testing with Apache JMeter
- Week 8 — White Box Testing
- Week 9 — Black Box Testing
- Week 10 — System Integration Testing (SIT)
- Week 11 — System Testing
- Week 12 — Testing & Risk
- Week 13 — Software Testing & Quality Management
- Week 14 — Software Testing Environment

# Week 1 — Introduction to Software Testing & SQA

---

## Main Ideas

This week introduces the foundations of the module: what software and quality mean, why testing matters, and how testing differs from debugging. It also introduces the vocabulary used throughout the course — error, fault, failure — and the cost and views of quality.

## Key Definitions

**Software:** Not just code — it also includes procedures, documentation, and the data needed to operate the system.

**Quality:** "Performance upon expectations" and "fit for functions."

**Software Quality (IEEE):** The degree to which a system, component, or process meets specified requirements AND meets customer/user needs or expectations.

**Software Quality Assurance (SQA) (IEEE):** A planned, systematic set of activities that gives confidence that a product conforms to established technical requirements; it evaluates the development process itself (contrast with Quality Control).

**Software Testing (Glenford Myers):** The process of executing a program with the intent of finding errors — testing activities run throughout the whole project life cycle, not just at the end.

**Reliability:** The probability that software will NOT cause system failure for a specified time under specified conditions.

## Why Testing Is Necessary

Poor-quality software can cause:

- Loss of money
- Loss of time
- Damage to business reputation
- Injury or even death (e.g., safety-critical systems like flight-traffic control)

Valid reasons to test include: learning about reliability, because testing is in the project plan, because failures are expensive, to avoid being sued, and to build confidence in quality. (Testing can never prove software has NO faults — that is a common misconception.)

## Errors, Faults and Failures

- **Error (mistake):** a human action that produces an incorrect result.
- **Fault / Defect / Bug:** the flaw in the code or work product introduced by an error.
- **Failure:** actual, observable incorrect behaviour that occurs when a faulty part of the code is executed (not every fault causes a failure every time).

### How an Error becomes a Failure



Figure 1.1 — An error leads to a fault, which may (or may not) lead to an observable failure.

Common causes of errors: time pressure, human fallibility, inexperienced staff, miscommunication about requirements, code/design complexity, unfamiliar technologies. Failures can also be caused purely by environmental conditions (not just code defects).

## Testing vs Debugging

- **Testing:** executes the software to reveal failures caused by defects.
- **Debugging:** a development activity that finds, analyses and fixes the defect; confirmation testing then checks the fix worked.

## Quality Control vs Quality Assurance

- **Quality Assurance (QA):** process-oriented; focuses on defect PREVENTION throughout development.
- **Quality Control (QC):** product-oriented; focuses on defect IDENTIFICATION in the finished product.

## Three Views of Quality



Figure 1.2 — Product, Process, and Business views of quality.

- **Product view:** functionality, correctness, performance, usability.
- **Process view:** efficiency, cost, schedule, rework caused by failures.
- **Business view:** timeliness and customer satisfaction — is it the right product, at the right time, at the right price?

## Cost of Quality

- **Prevention cost:** quality planning, formal technical reviews, testing equipment, training.
- **Appraisal cost:** in-process/inter-process inspection, testing.
- **Failure cost:** rework, repair, complaint resolution, product returns/replacement.

### Example: Errors, faults & failures in practice

- A developer misreads a requirement (ERROR) and writes 'if age > 18' instead of 'if age >= 18' (FAULT introduced in the code).
- The system runs and rejects an 18-year-old user who should be allowed to register (observable FAILURE).
- This shows how one small human mistake travels from error → fault → failure.

## Week 2 — Software Reviews

### Main Ideas

A software review is a manual static-testing activity: people examine a work product (requirements, design, code) to find and fix defects early. Reviews range from very casual (informal) to strictly structured (formal), and they can target technical content, management status, or independent compliance.

### Key Definition

*"A software review is a process or meeting during which a software product is examined by project personnel, users, or other interested parties for comment or approval."*

### Informal vs Formal Reviews

| Aspect           | Informal Review                 | Formal Review                      |
|------------------|---------------------------------|------------------------------------|
| Participants     | 2–3 people                      | 3–5 people                         |
| Scheduling       | Ad-hoc, during free time        | Pre-planned, gives time to prepare |
| Structure        | No planning, no guidance        | Structured; professional team      |
| Error correction | Not corrected during the review | Errors identified AND corrected    |
| Duration         | Unstructured                    | Does not exceed 2 hours            |
| Effectiveness    | Less effective                  | More effective                     |

### Review Meeting Outcomes (3 rules)

- Accept the model — meets expectations.
- Reject the model — too many errors; modify and hold another review.
- Accept with minor corrections — a few errors; correct them, no further review needed.

A formal technical review also produces a summary report answering: What was reviewed? Who reviewed it? What were the findings and conclusions?

### Types of Review

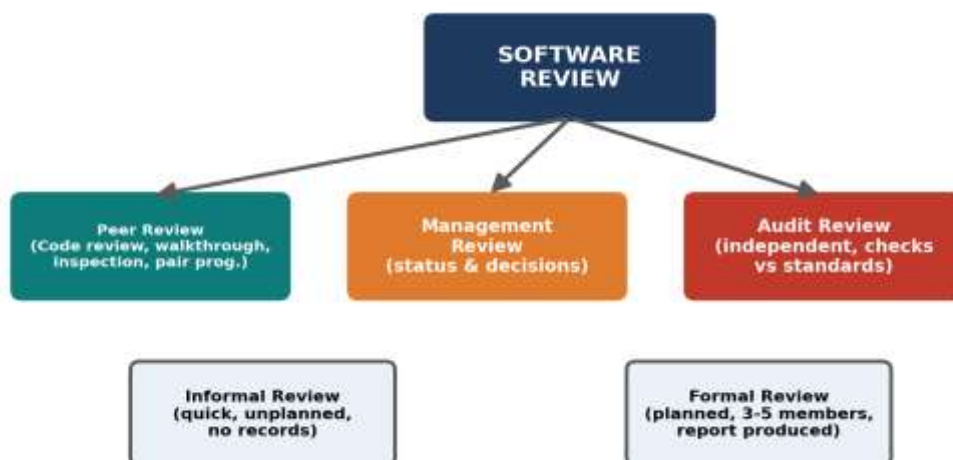


Figure 2.1 — Categories of software review.

- **1. Software Peer Review:** done by the author or colleagues on technical content/quality — includes Code Review, Pair Programming, Inspection, Walkthrough, and Technical Review.
- **2. Software Management Review:** conducted by management to evaluate status and make decisions.
- **3. Software Audit Review:** conducted by people OUTSIDE the project to check compliance with specifications/standards.

### Objectives of Software Review

- Improve development team productivity
- Make testing time- and cost-effective
- Produce final software with fewer defects
- Eliminate inadequacies

### Process of Software Review (5 steps)

- 1. Entry Evaluation — confirm documentation and entry requirements are met.
- 2. Management Preparation — assign roles, gather resources, brief management.
- 3. Review Planning — set goals/scope, invite parties, schedule the meeting.
- 4. Preparation — distribute resources, give reviewers time to prepare.
- 5. Examination & Exit Evaluation — reviewers collaborate, record concerns, agree remedial plans.

### Advantages of Reviews

- Defects found earlier (cheaper to fix)
- Reduces maintenance cost
- Trains technical authors
- Improves productivity, reduces timescales
- Reduces testing time/cost
- Improves quality and customer relations
- Cost-effective

#### Example: Informal vs formal review

- Informal: Two developers grab coffee and quickly glance over a colleague's login-page code — no notes are kept.
- Formal: A 4-person panel schedules a 90-minute inspection of the 'Payment Module' design document, follows a checklist, and produces a signed review report.

## Week 3 — Static Verification Techniques

### Main Ideas

This week separates VERIFICATION from VALIDATION, and STATIC testing (no code execution) from DYNAMIC testing (execution). Static testing catches defects very early — in requirements, design, and code — before any test is even run, making fixes far cheaper.

### Key Definitions

**Verification:** "Are we building the product RIGHT?" — does the software conform to its specification? Done via reviews, checklists, walkthroughs, inspections.

**Validation:** "Are we building the RIGHT product?" — does the software do what the user actually needs? Involves real execution/testing, and happens after verification.

**Static Testing (Analysis):** Reviewing/analysing documentation, design, or code WITHOUT executing the software. Good for proving correctness, but may give false positives.

**Dynamic Testing (Test/Experimentation):** Executing the software to find bugs.

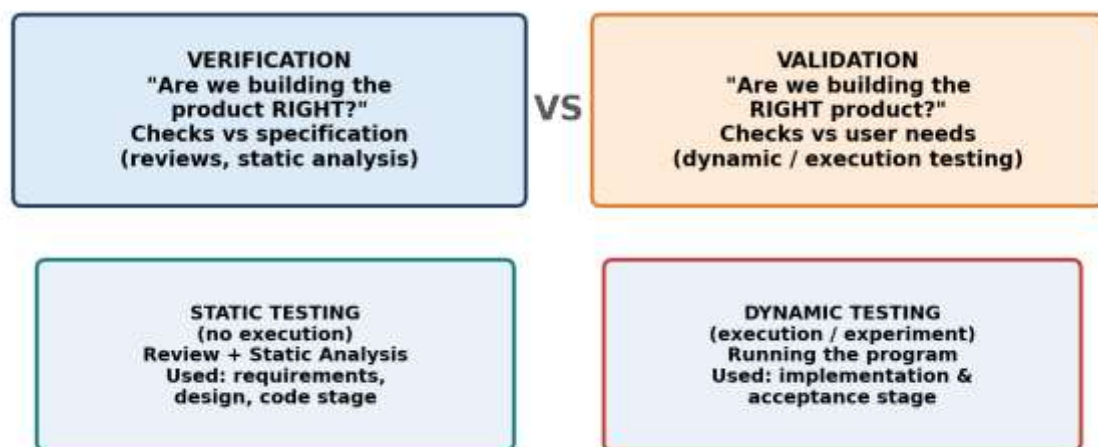


Figure 3.1 — Verification vs Validation, and Static vs Dynamic testing.

### When Static Testing Applies (Lifecycle)

| Lifecycle Phase                       | Dynamic Testing | Static Testing |
|---------------------------------------|-----------------|----------------|
| Requirements analysis & specification | —               | ✓              |
| Top-level design                      | —               | ✓              |
| Detailed design                       | —               | ✓              |
| Implementation                        | ✓               | ✓              |
| Acceptance testing                    | ✓               | —              |

### Two Basic Types of Static Testing

- 1. **People-based (Reviews):** manual — informal reviews, walkthroughs, technical reviews, inspections, audits.
- 2. **Tool-based (Static Analysis):** automated tools examine source code/bytecode for issues without running it.

## What Is Subject to Static Testing?

- Requirements & Specifications Review
- Structural Design Analysis
- Code Review
- Database Analysis
- Documentation Review
- User Manual Review

## Objectives of Static Testing

- Ensure programming conventions/standards are followed
- Verify source code compiles & builds
- Ensure design/implementation specs are met
- Identify logical, structural, formatting or syntactic errors

## Benefits of Static Testing

- Early defect detection (cheapest stage to fix)
- Improved code quality & maintainability
- Reduced cost and time
- Prevents common issues (syntax errors, null pointers, resource leaks)
- Improves team collaboration

## Manual vs Automated Static Testing

- **Manual:** Inspections, Walkthroughs, Informal reviews, Technical reviews, Audits.
- **Automated (Static Analysis tools):** Detect syntactic errors, logic errors, security vulnerabilities, code-quality issues, and coding-standard violations; also includes Lint checks and Formal Methods (mathematical proof of correctness).

Common tools: Checkstyle, SourceMeter, Soot, Lint, SonarQube, PMD.

### Example: Static vs dynamic testing

- **Static:** A reviewer reads the design document for an ATM system and notices the withdrawal-limit rule contradicts the account-type table — found WITHOUT running any code.
- **Dynamic:** A tester actually runs the ATM software and withdraws \$50 to see if the balance updates correctly.

## Week 4 — Software Testing Techniques (Black / White / Gray Box)

### Main Ideas

Because exhaustive testing is impossible, testers use structured techniques to design fewer, smarter test cases with better coverage. The three broad strategies differ by how much internal knowledge of the code the tester has.

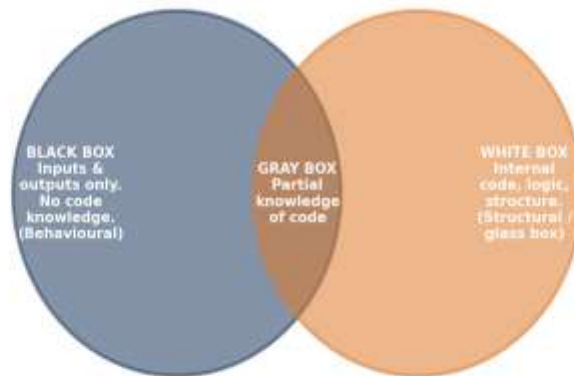


Figure 4.1 — Black Box, White Box, and Gray Box testing.

### Black Box Testing

Tester applies inputs and compares actual output with expected results, without knowing the internal code. Also called Behavioural Testing.

- **Techniques:** Boundary Value Analysis, Equivalence Class Partitioning, Decision Table testing, State Transition testing, Error guessing.

### White Box Testing

A verification technique examining internal paths, design, coding logic, structures, and implementation to check the code works as expected.

- **Techniques:** Statement Coverage, Branch Coverage, Path Coverage (detailed in Week 8).

### Gray Box Testing

Combines Black Box + White Box — testing is done with PARTIAL knowledge of the internal code.

- **Techniques:** Matrix Testing, Regression Testing, Orthogonal Array Testing, Pattern Testing.

#### Example: Choosing a technique

- **Black Box:** A tester who has never seen the code tries logging into a website with valid/invalid passwords.
- **White Box:** A developer writes unit tests that walk through every 'if' branch of the discount-calculation function.
- **Gray Box:** A tester who knows the database schema (but not the full source) tests whether an SQL query returns correct results for edge-case inputs.

## Week 5 — Test Case Design & Test Cases

### Main Ideas

A test case is the fundamental unit of testing work — it turns a requirement into a concrete, repeatable check. This week covers definitions, why documented test cases matter, their structure, and how to design and write good ones.

### Key Definitions

*"A set of test inputs, execution conditions, and expected results developed for a particular objective, such as to exercise a particular program path or verify compliance with a requirement. — IEEE 610 (1990)"*

*"Documentation specifying inputs, predicted results, and a set of execution conditions for a test item. — IEEE 829 (1983)"*

In simple terms: a test case describes an input/action/event and the expected response, used to check that a feature works correctly.

### Why We Need Documented Test Cases

- Ensures consistency in test execution
- Ensures better test coverage
- Depends on the process, not one person (survives staff turnover)
- Avoids retraining every new test engineer from scratch

### Types of Test Cases

- **Positive Test Case:** valid data entered to confirm the system produces the correct result as per requirements.
- **Negative Test Case:** invalid data entered deliberately to confirm the system correctly rejects it.

### Elements of a Test Case

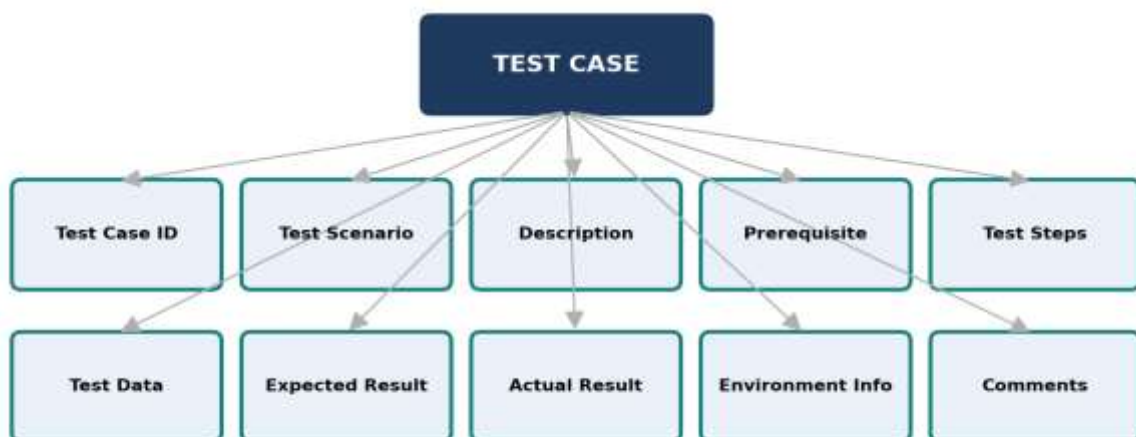


Figure 5.1 — The standard elements of a documented test case.

### Attributes of a Good Test Case

- Accurate

- Economical
- Repeatable & self-standing
- Appropriate
- Traceable (linked back to a requirement)

### Test Design Techniques — 3 Categories

- Specification-Based (Black Box) Design Techniques
- Structure-Based (White Box) Design Techniques
- Experience-Based Testing Techniques

Choosing a technique depends on: system type, identified risk, customer requirements, use-case models, and tester knowledge.

### 10 Best Practices for Writing Good Test Cases

- 1. Keep test cases simple and transparent
- 2. Design with the end user in mind
- 3. Avoid test case repetition
- 4. Do not assume — write it down
- 5. Aim for 100% coverage
- 6. Make test cases identifiable (unique IDs)
- 7. Apply proper testing techniques
- 8. Keep tests self-cleaning
- 9. Keep tests repeatable and self-standing
- 10. Get test cases peer reviewed

#### Example: Test case for 'Check Login Functionality'

- Positive: Login succeeds with a valid, registered username and correct password.
- Positive: Login succeeds after a password reset, using the new password.
- Negative: Login fails and shows an error with a blank username and password.
- Negative: Account locks after multiple failed login attempts.
- Negative: Login fails when the password case does not match (case-sensitive check).

## Week 6 — Test Automation

### Main Ideas

Automation testing replaces repetitive manual execution with tools/scripts that run tests and compare actual vs expected results with little human intervention. This week covers why/when to automate, the process, frameworks, and common tools — plus its risks.

### Key Definition

*"Automation testing is the practice of running tests automatically, executing them and comparing actual results with predicted results, with little or no intervention from the test engineer."*

### Why Automate?

- Saves money and time
- Vastly increases test coverage
- Improves accuracy
- Helps both developers and testers
- Does things manual testing cannot (e.g., huge repeated load)
- Increases speed of execution

### Manual vs Automation Testing

| Aspect            | Manual Testing                 | Automation Testing       |
|-------------------|--------------------------------|--------------------------|
| Performed by      | QA tester (human)              | Automation tools/scripts |
| Time & efficiency | Time-consuming, less efficient | Faster, more efficient   |
| Reliability       | Less reliable                  | More reliable            |
| Setup             | More straightforward           | More complex setup       |
| Programming       | Not required                   | Required                 |

### Best Candidates for Automation

- High-risk, business-critical test cases
- Repeatedly executed tests
- Time-consuming tests
- Tests difficult to perform manually

### Automation Testing Process



Figure 6.1 — The 5-stage automation testing process.

- **1. Test Tool Selection:** consider budget, price, OS/device support, technology support, data-source connectivity, reporting, integration with bug/test management repositories.
- **2. Define Scope of Automation:** business scenarios, manual test case coverage, common functionality, technical feasibility, reusability, end-to-end flows.
- **3. Planning, Design & Development:** tool selection, framework design, in/out-of-scope items, testbed prep, scheduling.
- **4. Test Execution:** run scripts with prepared test data using the automation/test-management tool; generate reports.
- **5. Maintenance:** review and update scripts as the application changes, to keep them effective.

## Automation Frameworks

- Data-Driven Automation Framework
- Keyword-Driven Automation Framework
- Modular Automation Framework
- Hybrid Automation Framework

## Testable Types & Tools

Testable with automation: Unit, Integration, Functional, Keyword, Regression, Data-Driven, Black-box, and Smoke testing.

- **Commercial tools:** IBM Rational Functional Tester, Katalon Studio, Ranorex, Sahi Pro, Apache JMeter, Appium, RedwoodHQ, Cypress.
- **Open-source tools:** Citrus, Galen, Karate-DSL, Robot Framework, Selenium, Watir, Gauge.

## The 'Dark Side' of Automation

- Automating the wrong things
- Tools that can't solve your real problems
- Tools that don't fit your testers' skill set
- Ignoring total cost of tool ownership
- Choosing a tool just because it is open source
- No testing culture that embraces automation

### Example: When (not) to automate

- Good candidate: The regression suite for the login page, run before every release — repetitive and business-critical.
- Poor candidate: A brand-new, rarely used admin screen that changes its UI weekly — high maintenance cost for little benefit.

## Week 7 — Performance Testing with Apache JMeter

---

### Main Ideas

Apache JMeter is a free, open-source, 100% Java tool for load, performance, functional, regression and stress testing across many protocols. This week covers its history, protocol support, core building blocks and how a test plan executes.

### Key Facts

**JMeter:** An open-source, pure-Java application originally written by Stefano Mazzocchi (Apache Software Foundation) to test the performance of Apache JServ (now Apache Tomcat); later redesigned with a GUI and functional-testing features.

### Protocols Supported

- Web – HTTP, HTTPS
- Web services – SOAP / REST
- FTP services
- Database via JDBC drivers
- Directory – LDAP
- Message-oriented middleware via JMS
- Mail – SMTP(S), POP3(S), IMAP(S)
- Native commands / shell scripts
- TCP

### Key Features

- Multithreading framework
- 100% Java scripted
- Pluggable & scriptable samplers
- GUI design and interface
- Data analysis & visualisation of results
- Highly extensible core

### JMeter Test-Plan Structure

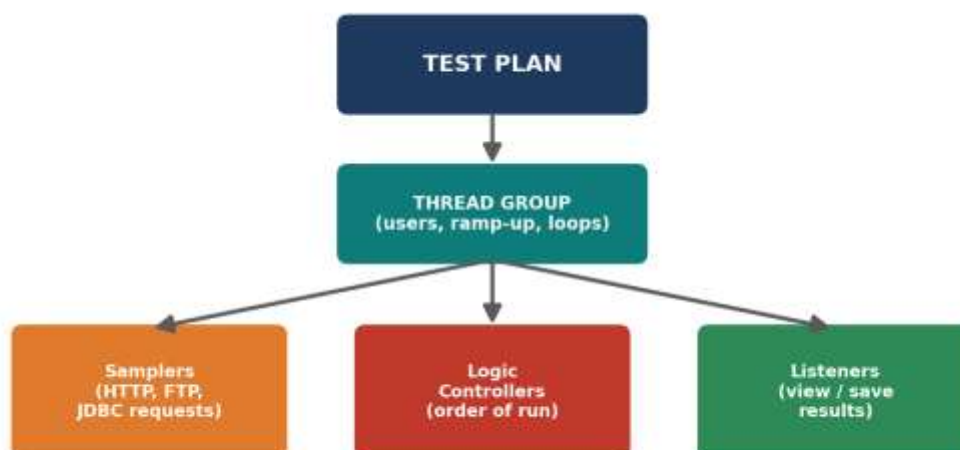


Figure 7.1 — How a JMeter Test Plan is organised.

- **Thread Group:** the starting point of a test plan — sets number of threads (users), ramp-up period, and loop/execution count.
- **Samplers:** send specific requests to a server, simulating a real user request.
- **Logic Controllers:** control the ORDER in which Samplers under them are processed.
- **Listeners:** view, save, and visualise the results of the test run.

### Execution Order Inside a Thread

- 1. Configuration elements
- 2. Pre-Processors
- 3. Timers
- 4. Sampler
- 5. Post-Processors
- 6. Assertions
- 7. Listeners

### Installation Basics

- Requires Java JDK 1.6+
- Set JAVA\_HOME
- Download JMeter from [jmeter.apache.org](http://jmeter.apache.org)
- Set JMETER\_HOME and add it to PATH
- Run via command line or GUI

#### Example: Typical JMeter use case

- Simulate 500 concurrent users logging into an e-commerce site (Thread Group = 500 threads, ramp-up 60s).
- An HTTP Request Sampler sends the login request to the server for each simulated user.
- A Summary Report Listener then shows average response time, error %, and throughput to judge if the server can handle real-world load.

## Week 8 — White Box Testing

---

### Main Ideas

White Box Testing (a.k.a. glass box, clear box, structural, transparent, or open-box testing) examines the internal structure, logic and code of the software rather than only its outputs. This week goes deep into the coverage techniques used to design white-box test cases.

### Key Definition

*"White box testing is a verification technique used by engineers to examine if the code works as expected, based on internal paths, design, coding, logic, and structures."*

### Process of White Box Testing

- **Input:** requirements, functional specs, design documents, source code.
- **Processing:** risk analysis guides the whole process; test planning designs cases that cover the entire code; execute, fix, and repeat until error-free.
- **Output:** a final report of the entire testing process.

### Coverage Techniques

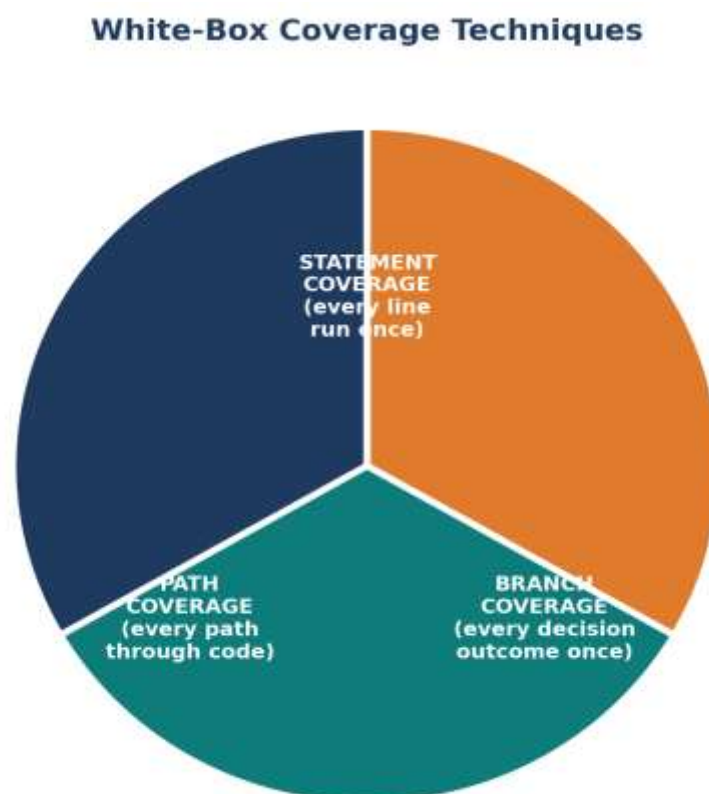


Figure 8.1 — The three core white-box coverage techniques.

- **1. Statement Coverage:** aim to execute every statement/line of code at least once (every node in a flowchart traversed at least once).
- **2. Branch Coverage:** design test cases so every branch from every decision point is traversed at least once (every edge in a flowchart).

- **3. Condition Coverage:** every individual Boolean sub-condition must evaluate to both TRUE and FALSE at least once.

### Condition Coverage worked example

READ X, Y IF (X == 0 || Y == 0) PRINT '0'

- TC1 — X = 0, Y = 5 (covers X == 0 → TRUE)
- TC2 — X = 5, Y = 0 (covers Y == 0 → TRUE)

Together these two test cases exercise both conditions being true at least once.

### Other White Box Techniques

- Multiple Condition Coverage — all combinations of conditions
- Basic Path Testing — tests every independent path through the code
- Loop Testing — Simple loops, Nested loops, Concatenated loops

White box testing is generally performed in 2 steps: (1) the tester must understand the code well, and (2) the tester writes and executes code-level test cases.

### Tools

Fiddler, JUnit.net, OpenGrok, Wireshark, HP Fortify, CSUnit, PyUnit, Sqlmap, Nmap, Parasoft Jtest, NUnit, VeraUnit, CppUnit, Bugzilla.

### Advantages vs Disadvantages

| Advantages                                                                                                                                                                                                           | Disadvantages                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Thorough testing of internal logic                                                                                                                                                                                   | Requires programming knowledge & source access  |
| Enables code optimisation                                                                                                                                                                                            | Risk of overemphasising internal workings       |
| Early detection of defects                                                                                                                                                                                           | Possible bias since tester may be the developer |
| Integrates well with the SDLC                                                                                                                                                                                        | High overhead in designing test cases           |
| Finds complex, hidden defects                                                                                                                                                                                        | Cannot detect MISSING functionality             |
| <b>Example:</b><br><b>Statement vs branch coverage</b> <ul style="list-style-type: none"> <li>• Code: IF (age &gt;= 18) THEN allow = true ELSE allow = false</li> <li>• Statement coverage needs only ONE</li> </ul> |                                                 |

| Advantages                                                                                                                                                                                                                                                                                                  | Disadvantages |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <p>test (e.g.,<br/>age = 20)<br/>to run<br/>every line.</p> <ul style="list-style-type: none"><li>• Branch<br/>coverage<br/>needs<br/>TWO tests<br/>— one for<br/>age &gt;= 18<br/>(true<br/>branch)<br/>and one<br/>for age &lt;<br/>18 (false<br/>branch) —<br/>to cover<br/>both<br/>outcomes.</li></ul> |               |

## Week 9 — Black Box Testing

### Main Ideas

Black Box Testing validates functionality purely against specifications/requirements, without any knowledge of the internal code. This week reviews its types, techniques, advantages/disadvantages, and directly compares it with White Box Testing.

### Key Definition

*"Black-box testing focuses on validating functionality based on specifications or requirements, without concern for internal implementation details."*



Figure 9.1 — The Black Box testing model: only inputs and outputs are visible.

### Types of Black Box Testing

- Functional Testing
- Regression Testing
- Non-Functional Testing (NFT)

### What Black Box Testing Can Find

- Missing functions, incorrect functions, interface errors
- Errors accessing the database
- Errors initiating or terminating functions
- Errors in performance or behaviour

### Techniques

- Boundary Value Analysis
- Equivalence Class Partitioning
- Decision Table based testing
- State Transition testing
- Error guessing

### Advantages vs Disadvantages

| Advantages                      | Disadvantages                            |
|---------------------------------|------------------------------------------|
| No programming knowledge needed | Same tests may be repeated unnecessarily |

| Advantages                               | Disadvantages                             |
|------------------------------------------|-------------------------------------------|
| Efficient for large systems              | Hard to write cases without clear specs   |
| Tests from the user's/client's viewpoint | Complex inputs make execution difficult   |
| Test cases easily reproducible           | Root cause of failure may be hard to find |
| Finds ambiguity in functional specs      | Does not reveal control-structure errors  |

## Tools

Appium, Selenium, Microsoft Coded UI, Applitools, HP QTP.

## Black Box vs White Box Testing

| Area             | Black Box Testing                             | White Box Testing                           |
|------------------|-----------------------------------------------|---------------------------------------------|
| Objective        | Functionality vs requirements                 | Correctness/efficiency of internal code     |
| Knowledge needed | None of internal workings                     | Programming, architecture, design           |
| Methods          | Equivalence partitioning, BVA, error guessing | Control-flow, data-flow, statement coverage |
| Scope            | Functional level                              | Unit, integration, and system level         |

### Example: Black box in action

- Testing a calculator app:  
enter 5 + 3 and check the output is 8 — the tester never looks at how addition is coded internally.
- Boundary Value Analysis  
example:  
for an age field accepting 18–60, test 17, 18, 60, and 61.

## Week 10 — System Integration Testing (SIT)

### Main Ideas

SIT checks that two or more integrated systems/modules work together correctly — verifying interactions, data flow, and compatibility once individual units have already passed their own tests.

### Key Definition

*"System Integration Testing is a QA process used to ensure the compatibility of two or more systems, confirming their interactions are appropriate and safe."*

### Importance of SIT

- Identifies and cleans up systems, data and process issues
- Verifies all components interact correctly
- Confirms changes to one component don't break others
- Identifies compatibility issues between systems
- Helps uncover security risks from component interaction

### Integration Testing Approaches

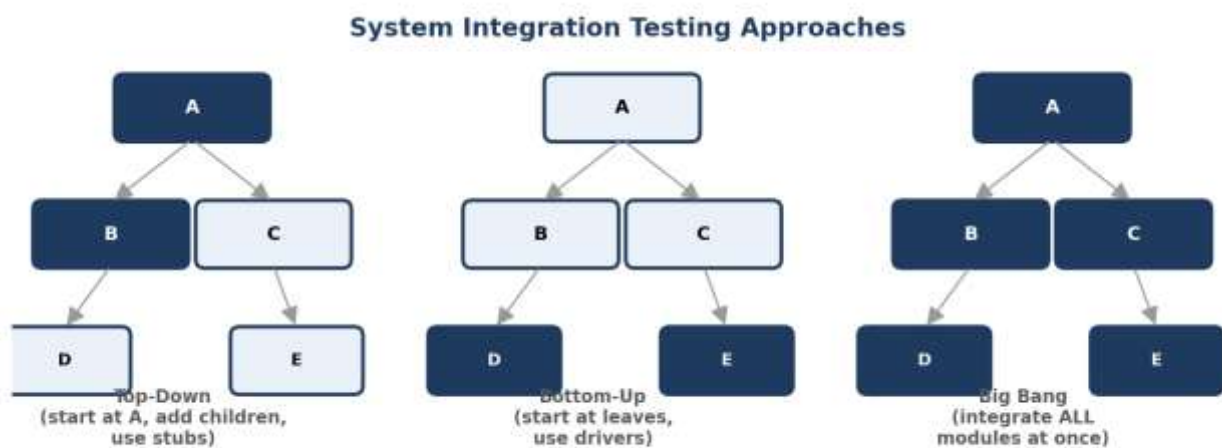


Figure 10.1 — Top-Down, Bottom-Up, and Big Bang integration approaches.

- **Top-Down Approach:** start testing from the top-level module downward, using STUBS to stand in for lower modules not yet integrated.
- **Bottom-Up Approach:** start testing from the lowest-level modules upward, using DRIVERS to simulate higher modules.
- **Big Bang Approach:** integrate ALL modules at once and test the system as a whole.
- **Incremental Integration Testing:** integrate and test modules one (or a few) at a time, gradually building up the system.

### Benefits of SIT

- Improved quality
- Increased efficiency
- Improved scalability

## Best Practices

- Establish clear goals
- Define test cases
- Design test data
- Automate tests where possible
- Monitor system performance
- Document results
- Re-test after fixes & updates

## Factors to Consider

- Interoperability between systems/components
- Data integrity across the integration
- Scalability of the integrated system
- Unavailability of systems or components

## Objectives of SIT

- Meet software user requirements
- Maximise memory usage efficiency
- Control data flow within the system
- Test control flow of the system
- Find errors and bugs
- Minimise time consumption of testing

## Three Major States of SIT

- **Integration layer:** checks data transformation between web services/middleware, using validation checkpoints.
- **Database layer:** checks data is transformed correctly into the database; validates data properties (mainly via SQL).
- **Application layer:** checks the data map between the database and the user interface.

### Example: Integration approach in practice

- An e-commerce system: the Payment module (leaf) is finished before the Order module (parent) — testers use Bottom-Up integration with a DRIVER simulating the Order module to test Payment early.
- Conversely, if the Checkout Page (top module) is ready first, testers use Top-Down integration with STUBS standing in for the not-yet-built Payment and Shipping modules.

## Week 11 — System Testing

---

### Main Ideas

System Testing evaluates the COMPLETE, fully integrated software as a whole, confirming it meets specified requirements. It happens after integration testing and before acceptance testing.

### Key Definition

*"System testing evaluates the overall functionality and performance of a complete and fully integrated software solution, checking readiness for delivery."*

### System Testing Process



Figure 11.1 — The system testing process flow.

- Test Environment Setup — build the environment for quality testing.
- Create Test Case — generate test cases for the process.
- Create Test Data — generate the data to be tested.
- Execute Test Case — run the prepared test cases.
- Defect Reporting — log detected defects; run Regression Testing to check side-effects; if unsuccessful, Retest.

### Objectives of System Testing

- Reduce risk
- Verify functional & non-functional behaviour matches design
- Validate the system is complete and works as expected
- Build confidence in overall quality
- Find defects and prevent them escaping to production

### Types of System Testing

- Regression Testing
- Load Testing
- Functional Testing
- Recovery Testing
- Migration Testing
- Usability Testing
- Software & Hardware (compatibility) Testing

#### Performance Testing sub-types

- **Load Testing:** behaviour under real-time/extreme load.

- **Stress Testing:** robustness under varying/extreme loads.
- **Scalability Testing:** ability to scale up/down with user load.

## Why System Testing Is Important

- Gives end-to-end assurance of system performance
- Tests both architecture and business requirements
- Helps mitigate live issues after production
- Compares old vs new system behaviour using the same data

## Tools

JMeter, Galen Framework, Selenium, HP Quality Center/ALM, IBM Rational Quality Manager, Appium, LoadRunner, SoapUI.

## Advantages vs Disadvantages

| Advantages                                                                                                                                                                                                                                                                            | Disadvantages                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| No deep programming knowledge required                                                                                                                                                                                                                                                | Time-consuming (tests the entire product)        |
| Tests the whole product; catches integration-level defects                                                                                                                                                                                                                            | High cost — covers entire software               |
| Environment mirrors real production use                                                                                                                                                                                                                                               | Needs good debugging tools to find hidden errors |
| Covers technical & business requirements                                                                                                                                                                                                                                              | Limited visibility into internal workings        |
| <b>Example: System testing scenario</b> <ul style="list-style-type: none"> <li>● A banking application has completed unit and integration testing. System testing now runs full end-to-end scenarios: login → transfer funds → check balance → logout, verifying the whole</li> </ul> |                                                  |

| Advantages                                              | Disadvantages |
|---------------------------------------------------------|---------------|
| workflow performs correctly under realistic conditions. |               |

## Week 12 — Testing & Risk

### Main Ideas

Risk-based testing focuses testing effort where it matters most. This week defines risk and its attributes, distinguishes Project Risk from Product Risk, and covers the broader discipline of Risk Management.

### Key Definitions

**Risk:** A potential event, hazard, threat, or situation whose occurrence causes an adverse effect.

- **Risk Likelihood:** the probability the risk occurs (between 0 and 1).
- **Risk Impact (harm):** the consequence if the risk occurs.

### Project Risk vs Product Risk

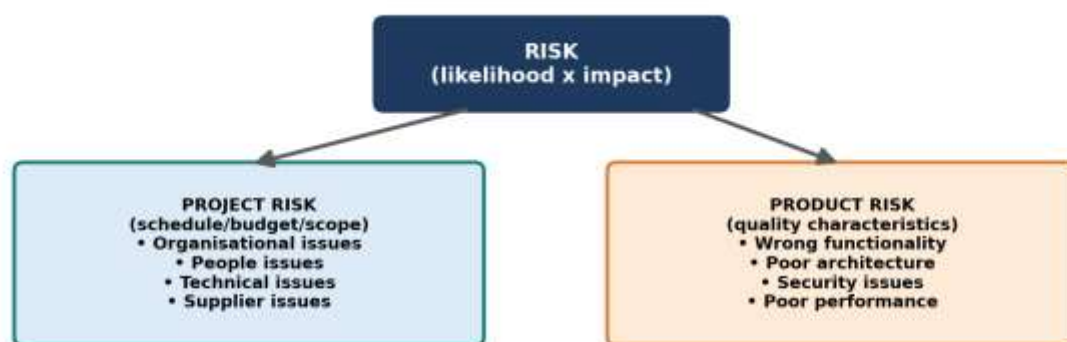


Figure 12.1 — Project risk vs product risk.

- **Project Risks:** relate to managing/controlling the project — organisational issues (delays, cost-cutting), people issues (skills, conflict, staff shortage), technical issues (scope creep, poor tools), supplier issues (third-party failure).
- **Product Risks:** relate to product quality characteristics (ISO 25010) — missing/wrong functionality, incorrect calculations, runtime errors, poor architecture, inefficient algorithms, slow response time, poor UX, security flaws.

Product risks, if they occur, can cause: user dissatisfaction, loss of revenue/trust/reputation, damage to third parties, high maintenance costs, criminal penalties, or even physical injury.

### Product Risk Analysis

- **Risk Identification:** generate a comprehensive list of risks (brainstorming, workshops, interviews, cause-effect diagrams).
- **Risk Assessment:** categorise risks, determine likelihood/impact/level, prioritise, and propose handling — using a quantitative approach (likelihood × impact) or a qualitative risk matrix.

Results of risk analysis help determine: scope of testing, test levels/types, techniques and coverage, effort estimation, prioritisation, and any non-testing risk-reduction activities.

### Product Risk Control

- **Risk Mitigation:** implement actions from risk assessment to reduce the risk level.
- **Risk Monitoring:** ensure mitigation actions are effective and identify emerging risks.

Response options: risk mitigation by testing, risk acceptance, risk transfer, or a contingency plan.

Mitigation-by-testing actions: choose testers with the right skills, apply appropriate independence of testing, conduct reviews/static analysis, use appropriate techniques/coverage, and perform dynamic + regression testing.

## Risk Management

*"Risk Management is a systematic process of recognising, evaluating, and handling threats or risks that affect an organisation's finances, capital, and operations."*

### Risk Management Process

- 1. Risk Identification
- 2. Risk Assessment (Analysis)
- 3. Risk Planning
- 4. Risk Monitoring

### Standards & Frameworks

- **COSO ERM Framework:** 2004, updated 2017 — 20 principles across 5 components, integrates risk into strategy.
- **ISO 31000:** 2009, revised 2018 — principles & framework for enterprise risk management.
- **BS 31100:** British Standard structured around ISO 31000 principles.

## Benefits and Limitations of Risk Management

| Benefits                                                                                                                                                                                                                                                                                | Limitations                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| Protects against potential losses                                                                                                                                                                                                                                                       | Too much focus on risk can miss opportunities       |
| Improves decision-making                                                                                                                                                                                                                                                                | Can be expensive to implement                       |
| Reduces unexpected expenses                                                                                                                                                                                                                                                             | Risk models can be overly complex                   |
| Ensures legal/regulatory compliance                                                                                                                                                                                                                                                     | Controls may create false sense of safety           |
| Builds resilience & safeguards reputation                                                                                                                                                                                                                                               | Relies on human judgement; some risks unpredictable |
| <b>Example: Project risk vs product risk</b> <ul style="list-style-type: none"><li>• Project risk: the lead developer resigns mid-project, causing a schedule delay (a PEOPLE issue).</li><li>• Product risk: the checkout page has a flaw that miscalculates tax, leading to</li></ul> |                                                     |

| Benefits                                                | Limitations |
|---------------------------------------------------------|-------------|
| financial loss for the company (a FUNCTIONALITY issue). |             |

# Week 13 — Software Testing & Quality Management

## Main Ideas

This week ties the whole module together by clearly separating Testing, Quality Assurance, and Quality Control, and introducing Software Quality Management (SQM) as the umbrella discipline that continuously improves product quality.

## Key Definitions

**Software Testing:** Detecting and solving technical issues in the source code, and assessing overall usability, performance, security, and compatibility.

**Software Quality:** The degree to which a product/system satisfies stated and implied customer/user needs, and meets industry quality standards.

**Quality Assurance (QA):** A PROACTIVE process starting at the beginning of development, aiming to PREVENT defects by setting quality/process standards and continuous improvement.

**Quality Control (QC):** The stage where the product is tested and reviewed to confirm it meets the quality standards set during quality planning; involves developers, testers, and sometimes users.

**Software Quality Management (SQM):** A systematic process of constant improvement of product quality — functionality, reliability and user satisfaction — integrated throughout development.

## QA vs QC vs Testing

QA contains QC contains Testing

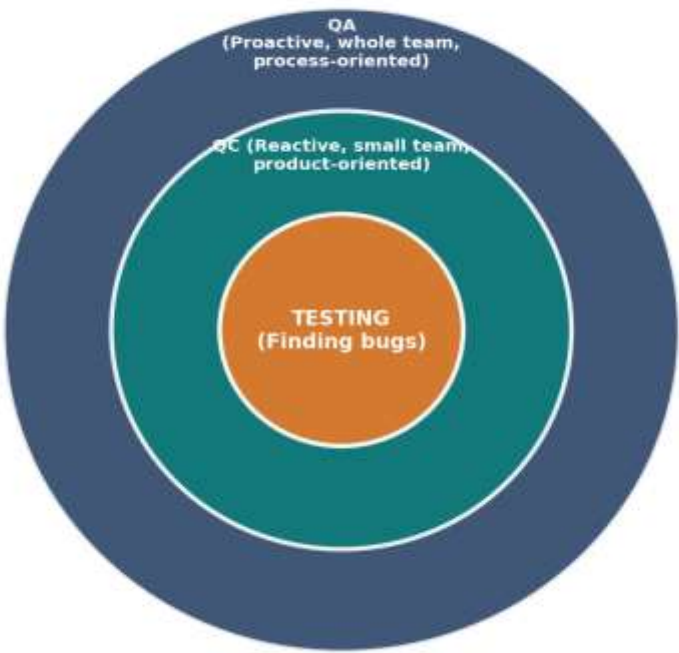


Figure 13.1 — QA contains QC, which contains Testing.

| Aspect   | QA        | QC       | Testing  |
|----------|-----------|----------|----------|
| Approach | Proactive | Reactive | Reactive |

| Aspect     | QA                                   | QC                                    | Testing                            |
|------------|--------------------------------------|---------------------------------------|------------------------------------|
| Focus      | Process-oriented                     | Product-oriented                      | Finding mistakes & bugs            |
| Techniques | Strategy, standards, test-mgmt plans | Verifying quality via testing/reviews | Manual or automated test execution |
| Timeframe  | Throughout development               | Before product release                | Along the development process      |
| Executor   | Entire team incl. stakeholders       | A small, dedicated team               | Test engineers, developers         |

## Why SQM Is Important

- Identifies and corrects errors, essential for effective project delivery
- Measures how well the product fulfils customer needs
- Measured by error frequency/repair, performance under conditions, and maintainability
- Compares actual results against expected results (completeness, efficiency, usability, correctness, reliability)

## Software Quality Management Metrics

- **Product metrics:** Mean Time to Failure, Defect Density, Customer Problems, Customer Satisfaction.
- **Process metrics:** Defect density during machine testing, defect arrival pattern, phase-based defect removal, defect removal effectiveness.
- **Project metrics:** Fix backlog & backlog management index, fix response time, % delinquent fixes, fix quality.

## Benefits of SQM

- Improves product quality
- Adds to customer satisfaction
- Meets user needs → higher adoption/revenue
- Reduces cost of rework via early defect detection
- Supports credible, reliable product decisions
- Increases productivity

### Example: QA vs QC vs Testing in one project

- QA: The team defines a coding standard and a peer-review process BEFORE writing any code (prevention).
- QC: Before release, a dedicated QC team verifies the final build meets the agreed quality checklist.
- Testing: A tester executes login/checkout test cases to find bugs in the running application.

## Week 14 — Software Testing Environment

### Main Ideas

A test environment is the hardware/software setup that lets testing teams run test cases and identify bugs before release. This week also introduces the different types of test environments and the career roles found in a QA/testing team.

### Key Definition

*"The test environment is the hardware and software set up for testing teams to run test cases; its configuration depends entirely on the requirements of the application under test."*

### Types of Test Environment

- **Integration Test Environment:** modules are integrated and their combined behaviour verified; should closely imitate production.
- **Performance Test Environment:** complex setup to test throughput, stability, response time under varying RAM/data volume — time-consuming and expensive.
- **Security Test Environment:** isolated environment (never touching production data) to test confidentiality, integrity, and authenticity.
- **Chaos Test Environment:** deliberately finds the specific area that can cause failure, before it affects real users.

### Key Areas to Set Up a Test Environment

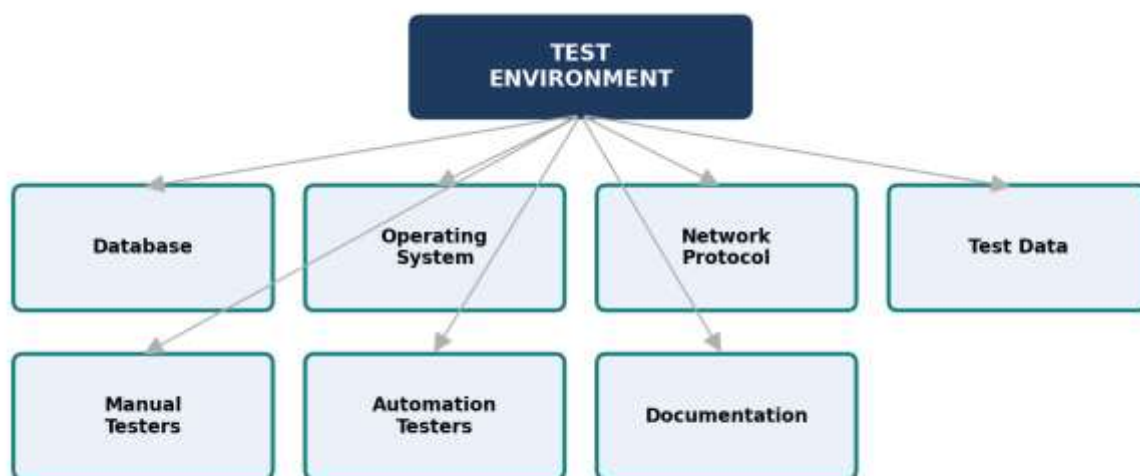


Figure 14.1 — Core components of a test environment.

- Database — needed at every part of the backend (client-server, mobile, or other).
- Operating System — client and server OS that manage every application.
- Network Protocol — LAN, wireless, or private network configurations as required.
- Test Data — complete, accurate, consistent data for designing effective tests.
- Manual Testers — check application quality and run tests manually.
- Automation Testers — build and run automated test scripts.
- Documentation — configuration/installation guides, user manuals.

## Process for Setting Up a Test Environment

- Test Server — some applications need a dedicated server (e.g., Java-based apps).
- Network — LAN/CAN/wireless setup that avoids congestion for other team members.
- PC Setup — different browsers/OS per tester as needed.
- Bug Reporting — a bug-tracking tool included in the environment.
- Test Tool — automation tool setup.
- Test Data — often copied from production (obfuscated/anonymised to protect privacy).

## Overview of Roles in Testing

A testing career typically has 7 major roles, based on knowledge and experience:

- 1. Junior Software Tester / QA Engineer
- 2. Senior Software Tester / Senior QA Engineer
- 3. Test Architect
- 4. QA Lead / Test Lead
- 5. QA Manager / Test Manager
- 6. Quality Head
- 7. Delivery Head

### Key Roles Explained

- **QA / Test Engineer:** designs, implements, executes and evaluates tests, and logs outcomes.
- **Test Lead:** handles test planning, defining required tests, monitoring progress, evaluating results.
- **Manual QA Engineer:** finds bugs through human observation, following a test plan, logging defects (e.g., in Jira).
- **Automation QA Engineer:** writes scripts using programming/automation-tool skills to automate repetitive test cases.
- **QA Lead:** supervises manual/automation engineers, administers resources, designs test programs and strategy.
- **QA Manager:** establishes QA procedures, ensures requirements/standards compliance, analyses reports, sets training needs, monitors performance.

### Example: Choosing a test environment

- Before a bank's mobile app release, the team sets up a Performance Test Environment mirroring production hardware to check response time under 10,000 simulated users, and a separate Security Test Environment using anonymised data to check for vulnerabilities — without ever touching real customer data.